

Contents

1	EXPERIMENT – 1	6
1.1	Virtual Machine and Container Setup	6
1.2	Part A: Virtual Machine using Vagrant (Windows)	6
1.2.1	Step 1: Install VirtualBox	6
1.2.2	Step 2: Install Vagrant	7
1.2.3	Verify Vagrant Installation	7
1.2.4	Step 3: Initialize Vagrant with Ubuntu Box	7
1.2.5	Start the Virtual Machine	7
1.2.6	Access the Virtual Machine	7
1.2.7	Step 4: Install Nginx inside the VM	8
1.2.8	Verify Nginx Installation	8
1.2.9	Utilization Matrix (Running State)	8
1.2.10	Stop the Virtual Machine	8
1.2.11	Remove the Virtual Machine	8
1.3	Part B: Container using WSL	8
1.3.1	Step 1: Install WSL 2	8
1.3.2	Step 2: Install Ubuntu on WSL	9
1.3.3	Step 3: Install Docker Engine inside WSL	9
1.3.4	Step 4: Run Ubuntu Container with Nginx	9
1.3.5	Step 5: Verify Nginx in Container	9
1.4	Result	10
2	Experiment 10: SonarQube — Static Code Analysis	10
2.1	Overview	10
2.2	Problem Statement	10
2.3	Architecture	11
2.3.1	Components:	11
2.4	Prerequisites	11
2.5	Step 1: Setup SonarQube using Docker	11
2.5.1	Run Containers	12
2.5.2	Check logs	13
2.5.3	Open in browser	13
2.6	Step 2: Generate Token	13
2.7	Step 3: Sample Java Project	13
2.7.1	Project Structure	13
2.7.2	Features with Issues:	13
2.8	Step 4: Maven Configuration	13
2.8.1	Create <code>Calculator.java</code>	14
2.8.2	Create <code>pom.xml</code>	14
2.9	Step 5: Run SonarQube Analysis	15
3	### Using Maven	15
3.1	Step 4: Run Scanner	15
3.2	Step 6: View Dashboard	15

4	Open:	15
4.1	Step 5: View Results	15
4.2	API Example	15
4.3	CI/CD Integration (Jenkins)	16
4.4	Key Concepts	16
4.5	Conclusion	16
5	EXPERIMENT – 2	17
5.1	Docker Installation, Configuration and Running Images	17
5.2	Software Requirements	17
5.3	Step-1: Pull Docker Image	17
5.4	Step-2: Run Container with Port Mapping	17
5.5	Step-3: Verify Running Containers	17
5.6	Step-4: Stop Container	18
5.7	Step-5: Remove Container	18
5.8	Step-6: Remove Docker Image	18
5.9	Result	18
5.10	Conclusion	18
6	Experiment 3: Docker – NGINX & Web Application Containerization	18
6.1	Objective	19
6.2	Experiment Structure	19
6.3	Part 1: Running Official NGINX Image	19
6.3.1	Pull Image	19
6.3.2	Run Container	19
6.3.3	Verify	19
6.4	Part 2: Custom NGINX Using Ubuntu Base Image	20
6.4.1	Dockerfile	20
6.4.2	Build Image	20
6.4.3	Run Container	20
6.5	Part 3: Custom NGINX Using Alpine Base Image	20
6.5.1	Dockerfile	20
6.5.2	Build Image	20
6.5.3	Run Container	20
6.5.4	Observation	21
6.6	Part 4: Image Size & Layer Comparison	21
6.6.1	Compare Images	21
6.6.2	Inspect Layers	21
6.6.3	Observations	21
6.7	Part 5: Serving Custom HTML Using Docker Volumes	21
6.7.1	Create HTML File	21
6.7.2	Run Container with Volume Mapping (Windows CMD)	21
6.8	Sample Web Application (Flask – Pre-Containerization)	21
6.8.1	app.py	21
6.8.2	requirements.txt	22

6.8.3	Run Locally	22
6.9	Tech Used	22
6.10	Experiment 4: Docker Essentials	22
7	Part 1: Containerizing Applications with Dockerfile	22
7.1	Step 1: Create Project Folder	22
7.2	Step 2: Create Application Files	23
7.2.1	app.py	23
7.2.2	requirements.txt	23
7.3	Step 3: Create Dockerfile	23
8	Part 2: Using .dockerignore	23
9	Part 3: Building Docker Images	24
9.1	Step 1: Build Image	24
9.2	Step 2: Tagging Images	24
9.3	Step 3: View Image Details	25
10	Part 4: Running Containers	25
10.1	Step 1: Run Container	25
10.2	Step 2: Manage Containers	26
11	Part 5: Multi-stage Builds	26
11.1	Why Multi-stage?	26
11.2	Dockerfile.multistage	26
11.3	Build and Compare	27
12	Part 6: Publishing to Docker Hub	27
12.1	Step 1: Login and Tag	27
12.2	Step 2: Pull & Run	27
13	Part 7: Node.js Example	28
13.1	Step 1: Create Node App	28
13.1.1	app.js	28
13.1.2	package.json	28
13.2	Step 2: Node Dockerfile	28
13.3	Step 3: Build & Run	29
14	Final Conclusion	29
15	Experiment 5: Docker - Volumes, Environment Variables, Monitoring & Networks	29
15.1	Objective	30
16	Part 1: Docker Volumes – Persistent Data Storage	30
16.1	Problem: Ephemeral Data	30
16.2	Volume Types	30

16.2.1	1. Anonymous Volumes	30
16.2.2	2. Named Volumes	30
16.2.3	3. Bind Mounts (Host Directory)	31
16.3	Practical Example: MySQL Persistence	31
17	Part 2: Environment Variables	31
17.1	Method 1: Using -e	31
17.2	Method 2: Using .env File	31
17.3	Method 3: In Dockerfile	32
17.4	Flask Example	32
17.5	Testing	32
18	Part 3: Docker Monitoring	32
18.1	Real-time Stats	32
18.2	Logs	32
18.3	Processes	32
18.4	Inspect Container	32
18.5	Events Monitoring	33
19	Part 4: Docker Networks	33
19.1	List Networks	33
19.2	Bridge Network	33
19.3	Host Network	33
19.4	None Network	33
19.5	Network Management	33
20	Part 5: Multi-Container Application	34
20.1	Architecture	34
20.2	Implementation	34
20.3	Monitoring	34
20.4	Connectivity Test	34
21	Key Concepts Learned	35
22	Conclusion	35
23	Experiment 6A: Comparison of Docker Run and Docker Compose	35
23.1	Objective	35
23.2	PART A - THEORY	35
23.2.1	1. Docker Run (Imperative Approach)	35
23.2.2	2. Docker Compose (Declarative Approach)	36
23.2.3	3. Mapping: Docker Run vs Compose	36
23.2.4	4. Advantages of Docker Compose	37
23.3	PART B - PRACTICAL TASKS	37
23.3.1	Task 1: Single Container	37
23.3.2	Task 2: Multi-Container (WordPress + MySQL)	38

23.4	PART C – CONVERSION TASKS	39
23.4.1	Task 3: Convert Docker Run to Compose	39
23.4.2	Task 4: Resource Limits	40
23.5	PART D – DOCKERFILE	40
23.5.1	Node App Example	40
23.6	Observations	41
23.7	Result	41
24	Lab Experiment 7: CI/CD using Jenkins, GitHub and Docker Hub	42
24.1	Aim	42
24.2	Objectives	42
24.3	Theory	42
24.3.1	1. What is Jenkins?	42
24.3.2	2. What is CI/CD?	42
24.3.3	3. Workflow Overview	43
24.3.4	4. Prerequisites	43
24.4	Part A: GitHub Repository Setup	43
24.4.1	1. Create Repository	43
24.4.2	2. Project Structure	43
24.4.3	3. Application Code	43
24.4.4	4. Dockerfile (Build Process)	44
24.4.5	Build Process Explanation	44
24.4.6	5. Jenkinsfile (Pipeline Definition)	44
24.5	Part B: Jenkins Setup using Docker	45
24.5.1	1. Create Docker Compose File	45
24.5.2	2. Start Jenkins	46
24.5.3	3. Unlock Jenkins	46
24.5.4	4. Initial Setup	46
24.6	Part C: Jenkins Configuration	46
24.6.1	1. Add Docker Hub Credentials	46
24.6.2	2. Create Pipeline Job	46
24.6.3	3. Run Pipeline	47
24.6.4	4. Verify in Docker Hub	47
24.7	Part D: GitHub Webhook Integration	47
24.7.1	Configure Webhook	47
24.8	Part E: Execution Flow	47
24.8.1	Stage 1: Code Push	47
24.8.2	Stage 2: Webhook Trigger	47
24.8.3	Stage 3: Jenkins Pipeline Execution	47
24.8.4	Stage 4: Artifact Ready	48
24.9	Role of Same Host Agent	48
24.9.1	Effect:	48
24.10	Observations	48
24.11	Result	48

Download VirtualBox

Figure 1: Download VirtualBox

25 Experiment 9: Ansible Automation using Docker (SSH-based Multi-Server Setup)	48
25.1 Overview	48
25.2 Problem Statement	49
25.3 Technologies Used	49
25.4 Architecture	49
25.5 Setup Steps	49
25.5.1 1. Create Docker Image	49
25.5.2 2. Run 4 Containers (Servers)	49
25.5.3 3. Verify Running Containers	50
25.5.4 4. SSH Test	50
25.6 Inventory File (inventory.ini)	50
25.7 Test Connectivity	50
25.7.1 Install dependency	50
25.7.2 Run Ansible Ping	50
25.8 Expected Output	50
25.9 Sample Playbook (update.yml)	51
25.10 Key Concepts Learned	51
25.11 Common Issues & Fixes	52
25.11.1.1. SSH Permission Denied	52
25.11.1.2. sshpass error	52
25.11.1.3. Host key checking error	52
25.12 Conclusion	52

1 EXPERIMENT – 1

1.1 Virtual Machine and Container Setup

1.2 Part A: Virtual Machine using Vagrant (Windows)

1.2.1 Step 1: Install VirtualBox

Download and install VirtualBox from the official website:

<https://www.virtualbox.org/wiki/Downloads>

Download Vagrant

Figure 2: Download Vagrant

Vagrant Version

Figure 3: Vagrant Version

1.2.2 Step 2: Install Vagrant

Download and install Vagrant from:

<https://developer.hashicorp.com/vagrant/install>

1.2.3 Verify Vagrant Installation

Check the installed version:

```
vagrant --version
```

1.2.4 Step 3: Initialize Vagrant with Ubuntu Box

Initialize Vagrant using Ubuntu:

```
vagrant init hashicorp/bionic64
```

1.2.5 Start the Virtual Machine

```
vagrant up
```

1.2.6 Access the Virtual Machine

```
vagrant ssh
```

Vagrant Init

Figure 4: Vagrant Init

Vagrant Up

Figure 5: Vagrant Up

Vagrant SSH

Figure 6: Vagrant SSH

1.2.7 Step 4: Install Nginx inside the VM

```
sudo apt update  
sudo apt install -y nginx  
sudo systemctl start nginx
```

1.2.8 Verify Nginx Installation

```
curl localhost
```

1.2.9 Utilization Matrix (Running State)

1.2.10 Stop the Virtual Machine

```
vagrant halt
```

1.2.11 Remove the Virtual Machine

```
vagrant destroy
```

1.3 Part B: Container using WSL

1.3.1 Step 1: Install WSL 2

```
wsl --install
```

Verify Nginx

Figure 7: Verify Nginx

Running State Matrix

Figure 8: Running State Matrix

Vagrant Halt

Figure 9: Vagrant Halt

1.3.2 Step 2: Install Ubuntu on WSL

```
wsl --install -d Ubuntu
```

1.3.3 Step 3: Install Docker Engine inside WSL

```
sudo apt update
sudo apt install -y docker.io
sudo systemctl start docker
sudo usermod -aG docker $USER
```

Docker Install Docker Service Start Docker User Group

Logout and login again to apply group changes.

1.3.4 Step 4: Run Ubuntu Container with Nginx

```
docker pull nginx
docker run -d -p 8080:80 --name nginx-container nginx
```

Docker Pull Docker Run

1.3.5 Step 5: Verify Nginx in Container

```
curl localhost:8080
```

WSL Install

Figure 10: WSL Install

Ubuntu on WSL

Figure 11: Ubuntu on WSL

Nginx in Container

Figure 12: Nginx in Container

1.4 Result

Virtual machines were successfully created using Vagrant and containers were deployed using Docker inside WSL. Nginx was installed and verified in both environments.

2 Experiment 10: SonarQube — Static Code Analysis

2.1 Overview

This project demonstrates how to use **SonarQube** for static code analysis using Docker. It includes:

- SonarQube Server (analysis + dashboard)
 - PostgreSQL Database (data storage)
 - Sample Java application with intentional issues
-

2.2 Problem Statement

Code bugs and security issues are often discovered too late. Manual code reviews:

- Are slow
- Do not scale
- Are inconsistent

Solution: Use SonarQube to automatically detect:

- Bugs
 - Vulnerabilities
 - Code Smells
 - Technical Debt
-

2.3 Architecture

Your Code → Sonar Scanner → SonarQube Server → PostgreSQL DB → Dashboard

2.3.1 Components:

- **SonarQube Server** → Stores results, applies rules, shows dashboard
 - **Sonar Scanner** → Analyzes code and sends report
 - **PostgreSQL** → Stores analysis data
-

2.4 Prerequisites

- Docker
 - Docker Compose
 - Java (JDK 11+)
 - Maven
-

2.5 Step 1: Setup SonarQube using Docker

Create `docker-compose.yml`:

```
=====
# Experiment 10: SonarQube - Static Code Analysis

## Objective
```

To set up SonarQube using Docker Compose, perform static code analysis on a Java project, and

```
## Step 1: Start the SonarQube Server
```

We will use Docker Compose to start both the SonarQube server and its PostgreSQL database to

```
### Create a file called `docker-compose.yml`
```

```
```yaml
version: '3.8'
>>>>>> 935e3a6 (Added)
services:
 sonar-db:
 image: postgres:13
 container_name: sonar-db
 environment:
```

```

 POSTGRES_USER: sonar
 POSTGRES_PASSWORD: sonar
 POSTGRES_DB: sonarqube
 POSTGRES_HOST_AUTH_METHOD: trust
volumes:
 - sonar-db-data:/var/lib/postgresql/data
networks:
 - sonarqube-lab

sonarqube:
 image: sonarqube:lts-community
 container_name: sonarqube
 ports:
 - "9000:9000"
 environment:
 SONAR_JDBC_URL: jdbc:postgresql://sonar-db:5432/sonarqube
 SONAR_JDBC_USERNAME: sonar
 SONAR_JDBC_PASSWORD: sonar
 volumes:
 - sonar-data:/opt/sonarqube/data
 - sonar-extensions:/opt/sonarqube/extensions
=====
>>>>>> 935e3a6 (Added)
 depends_on:
 - sonar-db
 networks:
 - sonarqube-lab

volumes:
 sonar-db-data:
<<<<<< HEAD
 sonar-data:
 sonar-extensions:
=====
>>>>>> 935e3a6 (Added)

networks:
 sonarqube-lab:

```

### 2.5.1 Run Containers

```

docker-compose up -d
docker-compose logs -f sonarqube

```

Open in browser: <http://localhost:9000>

===== ### Start both containers

```
docker-compose up -d
```

### 2.5.2 Check logs

```
docker-compose logs -f sonarqube
```

Wait until SonarQube starts successfully.

### 2.5.3 Open in browser

```
http://localhost:9000
```

---

## 2.6 Step 2: Generate Token

1. Go to **My Account** → **Security**
  2. Generate token (e.g., `scanner-token`)
  3. Copy it (shown only once)
- 

## 2.7 Step 3: Sample Java Project

### 2.7.1 Project Structure

```
sample-java-app/
 src/main/java/com/example/Calculator.java
 pom.xml
```

### 2.7.2 Features with Issues:

- Division by zero bug
  - SQL Injection vulnerability
  - Null pointer risk
  - Duplicate code
  - Unused variables
- 

## 2.8 Step 4: Maven Configuration

Add in `pom.xml`:

```
<properties>
 <sonar.projectKey>sample-java-app</sonar.projectKey>
 <sonar.host.url>http://localhost:9000</sonar.host.url>
 <sonar.login>YOUR_TOKEN</sonar.login>
</properties>
=====
```

Inside SonarQube dashboard:

- \* Click **Profile** (top-right)
- \* Go to **My Account** → **Security**
- \* Generate a token
- \* Save it for later use

---

## ## Step 3: Create Sample Java Project

```
```bash
mkdir -p sample-java-app/src/main/java/com/example
cd sample-java-app
```

2.8.1 Create Calculator.java

```
package com.example;

public class Calculator {
    public int add(int a, int b) {
        return a + b;
    }
}
```

2.8.2 Create pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
        http://maven.apache.org/xsd/maven-4.0.0.xsd">

    <modelVersion>4.0.0</modelVersion>
    <groupId>com.example</groupId>
    <artifactId>sample-java-app</artifactId>
    <version>1.0-SNAPSHOT</version>

    <build>
        <plugins>
            <plugin>
                <groupId>org.sonarsource.scanner.maven</groupId>
                <artifactId>sonar-maven-plugin</artifactId>
                <version>3.9.1.2184</version>
            </plugin>
        </plugins>
    </build>
```

</project>

2.9 Step 5: Run SonarQube Analysis

3 ### Using Maven

3.1 Step 4: Run Scanner

Inside the project folder:

```
mvn sonar:sonar -Dsonar.login=YOUR_TOKEN
```

3.2 Step 6: View Dashboard

4 Open:

Replace YOUR_TOKEN with the generated token.

4.1 Step 5: View Results

Open in browser:

```
http://localhost:9000
```

4.2 API Example

```
curl -u admin:YOUR_TOKEN \
"http://localhost:9000/api/issues/search?projectKeys=sample-java-app&types=BUG"
=====
```

You will see:

- * Code quality metrics
- * Bugs and vulnerabilities
- * Code smells
- * Analysis reports

Step 6: Stop the Server

```
```bash
docker-compose down
>>>>>> 935e3a6 (Added)
```

---

### 4.3 CI/CD Integration (Jenkins)

Pipeline Flow:

Checkout → Analyze → Quality Gate → Build → Deploy

Example snippet:

```
stage('SonarQube Analysis') {
 steps {
 withSonarQubeEnv('SonarQube') {
 sh 'mvn clean verify sonar:sonar'
 }
 }
}
```

---

### 4.4 Key Concepts

---

Term	Meaning
Bug	Code that may fail
Vulnerability	Security risk
Code Smell	Poor design
Technical Debt	Effort to fix issues
Quality Gate	Pass/Fail criteria

---

### 4.5 Conclusion

- SonarQube helps maintain high code quality
  - Scanner + Server both are required
  - Enables automated code review in CI/CD
-



## 5 EXPERIMENT – 2

### 5.1 Docker Installation, Configuration and Running Images

### 5.2 Software Requirements

- Operating System: Ubuntu / Linux
  - Docker Engine
- 

### 5.3 Step-1: Pull Docker Image

Pull a Docker image from Docker Hub.

```
docker pull nginx
```

Verify the image:

```
docker images
```

**Screenshot:** Step 1 – Pull Docker Image

---

### 5.4 Step-2: Run Container with Port Mapping

Run a container from the pulled image with port mapping.

```
docker run -d -p 8080:80 --name webserver nginx
```

**Screenshot:** Step 2 – Run Container

---

### 5.5 Step-3: Verify Running Containers

Check running containers and verify application access.

```
docker ps
```

Access in browser:

```
http://localhost:8080
```

**Screenshot:** Step 3 – Verify Running Container

---

## 5.6 Step-4: Stop Container

Stop the running container.

```
docker stop webserver
```

**Screenshot:** Step 4 – Stop Container

---

## 5.7 Step-5: Remove Container

Remove the stopped container.

```
docker rm webserver
```

**Screenshot:** Step 5 – Remove Container

---

## 5.8 Step-6: Remove Docker Image

Remove the Docker image from the system.

```
docker rmi nginx
```

**Screenshot:** Step 6 – Remove Image

---

## 5.9 Result

Docker was successfully installed and configured. Images were pulled, containers were run with port mapping, verified, stopped, removed, and Docker images were deleted successfully.

---

## 5.10 Conclusion

This experiment demonstrates Docker image management, container lifecycle management, and port mapping using Docker commands.

# 6 Experiment 3: Docker – NGINX & Web Application Containerization

This repository contains **Experiment-3**, demonstrating Docker fundamentals through NGINX deployment, custom image creation, image comparison, volume usage, and preparation of a sample Flask web application for containerization.

---

## 6.1 Objective

To understand and implement Docker containerization concepts by:

- Running official Docker images
  - Building custom Docker images using different base OS images
  - Comparing image sizes and layers
  - Serving custom content using Docker volumes
  - Preparing a simple web application for containerization
- 

## 6.2 Experiment Structure

EXPERIMENT-3/

```
Part 1/ # Official NGINX image
Part 2/ # Custom NGINX (Ubuntu base)
Part 3/ # Custom NGINX (Alpine base)
Part 4/ # Image comparison
Part 5/ # Custom HTML using volumes
Experiment 3 Part 2/
 flask-app/ # Sample Flask web application
 app.py
 requirements.txt
 Readme.md
```

---

## 6.3 Part 1: Running Official NGINX Image

### 6.3.1 Pull Image

```
docker pull nginx:latest
```

### 6.3.2 Run Container

```
docker run -d -p 8080:80 --name exp3-nginx nginx
```

### 6.3.3 Verify

```
curl http://localhost:8080
```

Default **NGINX welcome page** is displayed.

---

## 6.4 Part 2: Custom NGINX Using Ubuntu Base Image

### 6.4.1 Dockerfile

```
FROM ubuntu:22.04

RUN apt-get update && \
 apt-get install -y nginx && \
 apt-get clean && \
 rm -rf /var/lib/apt/lists/*

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
```

### 6.4.2 Build Image

```
docker build -t nginx-ubuntu .
```

### 6.4.3 Run Container

```
docker run -d -p 8081:80 --name nginx-ubuntu nginx-ubuntu
```

Custom Ubuntu-based NGINX image successfully created.

---

## 6.5 Part 3: Custom NGINX Using Alpine Base Image

### 6.5.1 Dockerfile

```
FROM alpine:latest

RUN apk add --no-cache nginx

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
```

### 6.5.2 Build Image

```
docker build -t nginx-alpine .
```

### 6.5.3 Run Container

```
docker run -d -p 8082:80 --name nginx-alpine nginx-alpine
```

#### 6.5.4 Observation

```
docker images nginx-alpine
```

Alpine-based image created (~16MB), demonstrating lightweight containers.

---

## 6.6 Part 4: Image Size & Layer Comparison

### 6.6.1 Compare Images

```
docker images | findstr nginx
```

### 6.6.2 Inspect Layers

```
docker history nginx
docker history nginx-ubuntu
docker history nginx-alpine
```

### 6.6.3 Observations

- Ubuntu image has multiple filesystem layers
  - Alpine image has minimal layers and smallest size
  - Official NGINX image is optimized but larger than Alpine
- 

## 6.7 Part 5: Serving Custom HTML Using Docker Volumes

### 6.7.1 Create HTML File

```
mkdir html
echo "<h1>Hello from Docker NGINX</h1>" > html/index.html
```

### 6.7.2 Run Container with Volume Mapping (Windows CMD)

```
docker run -d -p 8083:80 -v "%cd%\html:/usr/share/nginx/html" nginx
```

Custom HTML page served successfully via NGINX.

---

## 6.8 Sample Web Application (Flask – Pre-Containerization)

### 6.8.1 app.py

```
from flask import Flask
app = Flask(__name__)
```

## Step 1 Output

Figure 13: Step 1 Output

```
@app.route('/')
def hello():
 return 'Hello, Docker!'

if __name__ == '__main__':
 app.run(host='0.0.0.0', port=5000)
```

### 6.8.2 requirements.txt

flask

### 6.8.3 Run Locally

python app.py

Access:

http://localhost:5000

Application tested locally before containerization.

---

## 6.9 Tech Used

- Docker
- NGINX
- Ubuntu & Alpine Linux
- Python Flask

---

## 6.10 Experiment 4: Docker Essentials

# 7 Part 1: Containerizing Applications with Dockerfile

## 7.1 Step 1: Create Project Folder

```
mkdir my-flask-app
cd my-flask-app
```

## 7.2 Step 2: Create Application Files

### 7.2.1 app.py

```
from flask import Flask
app = Flask(__name__)

@app.route('/')
def hello():
 return "Hello from Docker!"

@app.route('/health')
def health():
 return "OK"

if __name__ == '__main__':
 app.run(host='0.0.0.0', port=5000)
```

---

### 7.2.2 requirements.txt

```
Flask==2.3.3
```

---

## 7.3 Step 3: Create Dockerfile

```
FROM python:3.10-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 5000

CMD ["python", "app.py"]
```

---

## 8 Part 2: Using .dockerignore

Create .dockerignore

Step 1 Output

Figure 14: Step 1 Output

Step 1 Output

Figure 15: Step 1 Output

```
__pycache__/
*.pyc
*.pyo
*.pyd
.env
.venv
env/
venv/
.vscode/
.idea/
.git/
.gitignore
.DS_Store
Thumbs.db
*.log
logs/
tests/
test_*.py
```

---

## 9 Part 3: Building Docker Images

### 9.1 Step 1: Build Image

```
docker build -t my-flask-app .
```

Check images:

```
docker images
```

---

### 9.2 Step 2: Tagging Images

Tag with version:

```
docker build -t my-flask-app:1.0 .
```

Multiple tags:



Step 1 Output

Figure 16: Step 1 Output

Step 1 Output

Figure 17: Step 1 Output

```
docker build -t my-flask-app:latest -t my-flask-app:1.0 .
```

Custom registry:

```
docker build -t ss1234ss1234/my-flask-app:1.0 .
```

Tag existing image:

```
docker tag my-flask-app:latest my-flask-app:v1
```

---

### 9.3 Step 3: View Image Details

```
docker images
```

```
docker history my-flask-app
```

```
docker inspect my-flask-app
```

Step 1 Output Step 1 Output —

## 10 Part 4: Running Containers

### 10.1 Step 1: Run Container

```
docker run -d -p 5000:5000 --name flask-container my-flask-app
```

Test:

```
curl http://localhost:5000
```

Check:

```
docker ps
```

```
docker logs flask-container
```

Step 1 Output Step 1 Output —

Step 1 Output

Figure 18: Step 1 Output

Step 1 Output

Figure 19: Step 1 Output

Step 1 Output

Figure 20: Step 1 Output

## 10.2 Step 2: Manage Containers

```
docker stop flask-container
docker start flask-container
docker rm flask-container
docker rm -f flask-container
```

---

# 11 Part 5: Multi-stage Builds

## 11.1 Why Multi-stage?

- Smaller image size
  - Better security
  - Separate build & runtime
- 

## 11.2 Dockerfile.multistage

```
FROM python:3.9-slim AS builder

WORKDIR /app

COPY requirements.txt .
RUN python -m venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"
RUN pip install --no-cache-dir -r requirements.txt

FROM python:3.9-slim

WORKDIR /app

COPY --from=builder /opt/venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"

COPY app.py .
```

### Step 1 Output

Figure 21: Step 1 Output

```
RUN useradd -m -u 1000 appuser
USER appuser

EXPOSE 5000

CMD ["python", "app.py"]
```

---

## 11.3 Build and Compare

```
docker build -t flask-regular .
docker build -f Dockerfile.multistage -t flask-multistage .
docker images | grep flask-
```

Step 1 Output Step 1 Output

---

## 12 Part 6: Publishing to Docker Hub

### 12.1 Step 1: Login and Tag

```
docker login
docker tag my-flask-app:latest ss1234ss1234/my-flask-app:1.0
docker tag my-flask-app:latest ss1234ss1234/my-flask-app:latest
```

Push:

```
docker push ss1234ss1234/my-flask-app:1.0
docker push ss1234ss1234/my-flask-app:latest
```

Step 1 Output Step 1 Output —

### 12.2 Step 2: Pull & Run

```
docker pull ss1234ss1234/my-flask-app:latest
docker run -d -p 5000:5000 ss1234ss1234/my-flask-app:latest
```

Step 1 Output Step 1 Output —

Step 1 Output

Figure 22: Step 1 Output

## 13 Part 7: Node.js Example

### 13.1 Step 1: Create Node App

```
mkdir my-node-app
cd my-node-app
```

#### 13.1.1 app.js

```
const express = require('express');
const app = express();
const port = 3000;

app.get('/', (req, res) => {
 res.send('Hello from Node.js Docker!');
});

app.get('/health', (req, res) => {
 res.json({ status: 'healthy' });
});

app.listen(port, () => {
 console.log(`Server running on port ${port}`);
});
```

---

#### 13.1.2 package.json

```
{
 "name": "node-docker-app",
 "version": "1.0.0",
 "main": "app.js",
 "dependencies": {
 "express": "^4.18.2"
 }
}
```

---

### 13.2 Step 2: Node Dockerfile

```
FROM node:18-alpine
```

## Step 1 Output

Figure 23: Step 1 Output

```
WORKDIR /app

COPY package*.json ./
RUN npm install --only=production

COPY app.js .

EXPOSE 3000

CMD ["node", "app.js"]
```

---

### 13.3 Step 3: Build & Run

```
docker build -t my-node-app .
docker run -d -p 3000:3000 --name node-container my-node-app
```

Test:

```
curl http://localhost:3000
```

---

## 14 Final Conclusion

This experiment covered:

- Dockerfile basics
- .dockerignore
- Image building & tagging
- Container management
- Multi-stage builds
- Docker Hub publishing
- Python & Node.js containerization

## 15 Experiment 5: Docker - Volumes, Environment Variables, Monitoring & Networks

---

## 15.1 Objective

To understand advanced Docker concepts including:

Data persistence using volumes Managing environment variables Monitoring containers Docker networking and multi-container communication

---

# 16 Part 1: Docker Volumes – Persistent Data Storage

## 16.1 Problem: Ephemeral Data

By default, container data is **temporary**. When a container stops or is removed, data is lost.

```
docker run -it --name test-container ubuntu /bin/bash
```

```
echo "Hello World" > /data/message.txt
```

```
cat /data/message.txt
```

```
exit
```

```
docker start test-container
```

```
docker exec test-container cat /data/message.txt
```

Data is lost after restart.

---

## 16.2 Volume Types

### 16.2.1 1. Anonymous Volumes

```
docker run -d -v /app/data --name web1 nginx
```

```
docker volume ls
```

```
docker inspect web1
```

Auto-created volume with random name

---

### 16.2.2 2. Named Volumes

```
docker volume create mydata
```

```
docker run -d -v mydata:/app/data --name web2 nginx
```

```
docker volume ls
```

```
docker volume inspect mydata
```

Persistent and reusable

---

### 16.2.3 3. Bind Mounts (Host Directory)

```
mkdir ~/myapp-data
docker run -d -v ~/myapp-data:/app/data --name web3 nginx

echo "From Host" > ~/myapp-data/host-file.txt
docker exec web3 cat /app/data/host-file.txt
```

Direct link between host and container

---

## 16.3 Practical Example: MySQL Persistence

```
docker run -d \
--name mysql-db \
-v mysql-data:/var/lib/mysql \
-e MYSQL_ROOT_PASSWORD=secret \
mysql:8.0
```

Data persists even after container removal.

---

## 17 Part 2: Environment Variables

### 17.1 Method 1: Using -e

```
docker run -d \
--name app1 \
-e DEBUG=true \
-e PORT=3000 \
my-node-app
```

---

### 17.2 Method 2: Using .env File

```
echo "DATABASE_HOST=localhost" > .env

docker run -d \
--env-file .env \
--name app2 \
my-app
```

---

### 17.3 Method 3: In Dockerfile

```
ENV NODE_ENV=production
ENV PORT=3000
```

---

### 17.4 Flask Example

Application reads environment variables:

```
import os
db_host = os.environ.get('DATABASE_HOST', 'localhost')
```

---

### 17.5 Testing

```
docker exec flask-app env
curl http://localhost:5001/config
```

---

## 18 Part 3: Docker Monitoring

### 18.1 Real-time Stats

```
docker stats
docker stats --no-stream
```

---

### 18.2 Logs

```
docker logs container-name
docker logs -f container-name
```

---

### 18.3 Processes

```
docker top container-name
```

---

### 18.4 Inspect Container

```
docker inspect container-name
```

---



## 18.5 Events Monitoring

```
docker events
docker events --filter 'event=start'
```

---

## 19 Part 4: Docker Networks

### 19.1 List Networks

```
docker network ls
```

---

### 19.2 Bridge Network

```
docker network create my-network

docker run -d --name web1 --network my-network nginx
docker run -d --name web2 --network my-network nginx

docker exec web1 curl http://web2
```

Containers communicate using names

---

### 19.3 Host Network

```
docker run -d --network host nginx
```

Uses host network directly

---

### 19.4 None Network

```
docker run -d --network none alpine sleep 3600
```

No internet access

---

### 19.5 Network Management

```
docker network create app-network
docker network connect app-network container-name
docker network rm network-name
```

---

## 20 Part 5: Multi-Container Application

### 20.1 Architecture

- Flask App
  - PostgreSQL Database
  - Redis Cache
- 

### 20.2 Implementation

```
docker network create myapp-network
```

```
docker run -d \
--name postgres \
--network myapp-network \
-e POSTGRES_PASSWORD=mysecretpassword \
-v postgres-data:/var/lib/postgresql/data \
postgres:15
```

```
docker run -d \
--name redis \
--network myapp-network \
redis:7-alpine
```

```
docker run -d \
--name flask-app \
--network myapp-network \
-p 5000:5000 \
-e DATABASE_URL="postgresql://postgres:mysecretpassword@postgres:5432/mydatabase" \
-e REDIS_URL="redis://redis:6379" \
flask-app
```

---

### 20.3 Monitoring

```
docker ps
docker stats
docker logs -f flask-app
```

---

### 20.4 Connectivity Test

```
docker exec flask-app ping postgres
docker exec flask-app ping redis
```

---

## 21 Key Concepts Learned

- Docker volumes enable **persistent storage**
  - Environment variables allow **dynamic configuration**
  - Monitoring tools help track **performance & logs**
  - Docker networks enable **container communication**
  - Multi-container apps simulate **real-world architecture**
- 

## 22 Conclusion

This experiment demonstrates how Docker supports:

- Persistent storage
  - Configurable environments
  - Monitoring and debugging
  - Scalable multi-container systems
- 

## 23 Experiment 6A: Comparison of Docker Run and Docker Compose

---

### 23.1 Objective

To understand the relationship between **docker run** and Docker Compose, and compare their syntax, features, and use cases.

---

### 23.2 PART A - THEORY

#### 23.2.1 1. Docker Run (Imperative Approach)

The **docker run** command is used to create and start a container from an image. It requires explicit flags:

- Port mapping (**-p**)
- Volume mounting (**-v**)
- Environment variables (**-e**)
- Network (**--network**)
- Restart policies (**--restart**)

- Resource limits (`--memory`, `--cpus`)
- Container name (`--name`)

This is an **imperative approach** (step-by-step execution).

#### 23.2.1.1 Example:

```
docker run -d \
 --name my-nginx \
 -p 8080:80 \
 -v ./html:/usr/share/nginx/html \
 -e NGINX_HOST=localhost \
 --restart unless-stopped \
 nginx:alpine
```

---

### 23.2.2 2. Docker Compose (Declarative Approach)

Docker Compose uses a YAML file (`docker-compose.yml`) to define services.

This is a **declarative approach** (desired state).

#### 23.2.2.1 Command:

```
docker compose up -d
```

#### 23.2.2.2 Equivalent Compose File:

```
version: '3.8'

services:
 nginx:
 image: nginx:alpine
 container_name: my-nginx
 ports:
 - "8080:80"
 volumes:
 - ./html:/usr/share/nginx/html
 environment:
 NGINX_HOST: localhost
 restart: unless-stopped
```

---

### 23.2.3 3. Mapping: Docker Run vs Compose

---

Docker Run	Docker Compose
-p	ports
-v	volumes
-e	environment
--name	container_name
--network	networks
--restart	restart
--memory	deploy.resources.limits.memory
--cpus	deploy.resources.limits.cpus

---



---

#### 23.2.4 4. Advantages of Docker Compose

- Multi-container support
- Easy reproducibility
- Version control friendly
- Simplified lifecycle management
- Supports scaling

```
docker compose up --scale web=3
```

---

## 23.3 PART B - PRACTICAL TASKS

### 23.3.1 Task 1: Single Container

#### 23.3.1.1 Using Docker Run

```
docker run -d \
 --name lab-nginx \
 -p 8081:80 \
 -v $(pwd)/html:/usr/share/nginx/html \
 nginx:alpine
```

Access:

```
http://localhost:8081
```

Stop:

```
docker stop lab-nginx
docker rm lab-nginx
```

---

#### 23.3.1.2 Using Docker Compose

```
version: '3.8'

services:
 nginx:
 image: nginx:alpine
 container_name: lab-nginx
 ports:
 - "8082:80"
 volumes:
 - ./html:/usr/share/nginx/html
```

Run:

```
docker compose up -d
```

Stop:

```
docker compose down
```

---

### 23.3.2 Task 2: Multi-Container (WordPress + MySQL)

#### 23.3.2.1 Using Docker Run

```
docker network create wp-net
```

```
docker run -d \
 --name mysql \
 --network wp-net \
 -e MYSQL_ROOT_PASSWORD=secret \
 -e MYSQL_DATABASE=wordpress \
 mysql:5.7
```

```
docker run -d \
 --name wordpress \
 --network wp-net \
 -p 8082:80 \
 -e WORDPRESS_DB_HOST=mysql \
 -e WORDPRESS_DB_PASSWORD=secret \
 wordpress:latest
```

Access:

```
http://localhost:8082
```

---

#### 23.3.2.2 Using Docker Compose

```

version: '3.8'

services:
 mysql:
 image: mysql:5.7
 environment:
 MYSQL_ROOT_PASSWORD: secret
 MYSQL_DATABASE: wordpress
 volumes:
 - mysql_data:/var/lib/mysql

 wordpress:
 image: wordpress:latest
 ports:
 - "8083:80"
 environment:
 WORDPRESS_DB_HOST: mysql
 WORDPRESS_DB_PASSWORD: secret
 depends_on:
 - mysql

volumes:
 mysql_data:

```

Run:

```
docker compose up -d
```

Stop:

```
docker compose down
```

---

## 23.4 PART C – CONVERSION TASKS

### 23.4.1 Task 3: Convert Docker Run to Compose

```

docker run -d \
 --name webapp \
 -p 5000:5000 \
 -e APP_ENV=production \
 -e DEBUG=false \
 --restart unless-stopped \
 node:18-alpine

```

Equivalent:

```
version: '3.8'
```

```
services:
 webapp:
 image: node:18-alpine
 container_name: webapp
 ports:
 - "5000:5000"
 environment:
 APP_ENV: production
 DEBUG: "false"
 restart: unless-stopped
```

---

### 23.4.2 Task 4: Resource Limits

```
docker run -d \
 --name limited-app \
 -p 9000:9000 \
 --memory="256m" \
 --cpus="0.5" \
 --restart always \
 nginx:alpine
```

Equivalent:

```
version: '3.8'
```

```
services:
 limited-app:
 image: nginx:alpine
 container_name: limited-app
 ports:
 - "9000:9000"
 restart: always
 deploy:
 resources:
 limits:
 memory: 256M
 cpus: "0.5"
```

---

## 23.5 PART D – DOCKERFILE

### 23.5.1 Node App Example

#### 23.5.1.1 app.js



```
const http = require('http');

http.createServer((req, res) => {
 res.write("Hello from Node Docker App");
 res.end();
}).listen(3000);
```

#### 23.5.1.2 Dockerfile

```
FROM node:18-alpine

WORKDIR /app
COPY app.js .

EXPOSE 3000
CMD ["node", "app.js"]
```

#### 23.5.1.3 docker-compose.yml

```
version: '3.8'

services:
 nodeapp:
 build: .
 container_name: custom-node-app
 ports:
 - "3000:3000"
```

Run:

```
docker compose up --build -d
```

Access:

```
http://localhost:3000
```

---

### 23.6 Observations

- `docker run` is simple for single containers
  - Docker Compose is better for multi-container setups
  - Compose improves readability and maintainability
- 

### 23.7 Result

Successfully compared Docker Run and Docker Compose and implemented real-world containerized applications.

---

## 24 Lab Experiment 7: CI/CD using Jenkins, GitHub and Docker Hub

---

### 24.1 Aim

To design and implement a complete CI/CD pipeline using Jenkins, integrating source code from GitHub, and building & pushing Docker images to Docker Hub.

---

### 24.2 Objectives

- Understand CI/CD workflow using Jenkins (GUI-based tool)
  - Create a structured GitHub repository with application + Jenkinsfile
  - Build Docker images from source code
  - Securely store Docker Hub credentials in Jenkins
  - Automate build & push process using webhook triggers
  - Use same host (Docker) as Jenkins agent
- 

### 24.3 Theory

#### 24.3.1 1. What is Jenkins?

Jenkins is a web-based GUI automation server used to:

- Build applications
- Test code
- Deploy software

It provides:

- Dashboard (browser-based UI)
  - Plugin ecosystem (GitHub, Docker, etc.)
  - Pipeline as Code using Jenkinsfile
- 

#### 24.3.2 2. What is CI/CD?

- **Continuous Integration (CI):** Code is automatically built and tested after each commit

- **Continuous Deployment (CD):** Built artifacts (Docker images) are automatically delivered/deployed
- 

### 24.3.3 3. Workflow Overview

Developer → GitHub → Webhook → Jenkins → Build → Docker Hub

---

### 24.3.4 4. Prerequisites

- Docker & Docker Compose installed
  - GitHub account
  - Docker Hub account
  - Basic Linux command knowledge
- 

## 24.4 Part A: GitHub Repository Setup

### 24.4.1 1. Create Repository

Create a repository on GitHub:

my-app

---

### 24.4.2 2. Project Structure

```
my-app/
 app.py
 requirements.txt
 Dockerfile
 Jenkinsfile
```

---

### 24.4.3 3. Application Code

#### 24.4.3.1 app.py

```
from flask import Flask

app = Flask(__name__)

@app.route("/")
def home():
 return "Hello from CI/CD Pipeline!"
```

```
app.run(host="0.0.0.0", port=80)
```

---

#### 24.4.3.2 requirements.txt

```
flask
```

---

#### 24.4.4 4. Dockerfile (Build Process)

```
FROM python:3.10-slim

WORKDIR /app
COPY . .

RUN pip install -r requirements.txt

EXPOSE 80
CMD ["python", "app.py"]
```

---

#### 24.4.5 Build Process Explanation

- Source code pushed to GitHub
  - Jenkins pulls code
  - Dockerfile:
    - Creates environment
    - Installs dependencies
    - Packages app
  - Output → Docker Image
- 

#### 24.4.6 5. Jenkinsfile (Pipeline Definition)

```
pipeline {
 agent any

 environment {
 IMAGE_NAME = "your-dockerhub-username/myapp"
 }

 stages {
```

```

 stage('Clone Source') {
 steps {
 git 'https://github.com/your-username/my-app.git'
 }
 }

 stage('Build Docker Image') {
 steps {
 sh 'docker build -t $IMAGE_NAME:latest .'
 }
 }

 stage('Login to Docker Hub') {
 steps {
 withCredentials([string(credentialsId: 'dockerhub-token', variable: 'DOCKER_TOKEN')]) {
 sh 'echo $DOCKER_TOKEN | docker login -u your-dockerhub-username --password-stdin'
 }
 }
 }

 stage('Push to Docker Hub') {
 steps {
 sh 'docker push $IMAGE_NAME:latest'
 }
 }
}

```

---

## 24.5 Part B: Jenkins Setup using Docker

### 24.5.1 1. Create Docker Compose File

```

version: '3.8'

services:
 jenkins:
 image: jenkins/jenkins:lts
 container_name: jenkins
 restart: always
 ports:
 - "8080:8080"
 - "50000:50000"
 volumes:
 - jenkins_home:/var/jenkins_home

```

```
 - /var/run/docker.sock:/var/run/docker.sock
 user: root

volumes:
 jenkins_home:
```

---

### 24.5.2 2. Start Jenkins

```
docker-compose up -d
```

Access Jenkins:

<http://localhost:8080/>

---

### 24.5.3 3. Unlock Jenkins

```
docker exec -it jenkins cat /var/jenkins_home/secrets/initialAdminPassword
```

---

### 24.5.4 4. Initial Setup

- Install suggested plugins
  - Create admin user
- 

## 24.6 Part C: Jenkins Configuration

### 24.6.1 1. Add Docker Hub Credentials

Path:

Manage Jenkins → Credentials → Add Credentials

- Type: Secret Text
  - ID: `dockerhub-token`
  - Value: Docker Hub Access Token
- 

### 24.6.2 2. Create Pipeline Job

- New Item → Pipeline
- Name: `ci-cd-pipeline`

Configure:

- Pipeline script from SCM

- SCM: Git
  - Repository URL: your GitHub repo
  - Script Path: `Jenkinsfile`
- 

### 24.6.3 3. Run Pipeline

Trigger build and monitor stages in Jenkins dashboard.

---

### 24.6.4 4. Verify in Docker Hub

Check if image is successfully pushed.

---

## 24.7 Part D: GitHub Webhook Integration

### 24.7.1 Configure Webhook

In GitHub:

Settings → Webhooks → Add Webhook

- Payload URL:

`http://<your-server-ip>:8080/github-webhook/`

- Events: Push events
- 

## 24.8 Part E: Execution Flow

### 24.8.1 Stage 1: Code Push

Developer updates code in GitHub

### 24.8.2 Stage 2: Webhook Trigger

GitHub sends event to Jenkins

### 24.8.3 Stage 3: Jenkins Pipeline Execution

- **Clone:** Pulls latest code
- **Build:** Docker builds image
- **Auth:** Jenkins logs into Docker Hub
- **Push:** Image pushed to Docker Hub

#### 24.8.4 Stage 4: Artifact Ready

- Docker image available globally
- 

### 24.9 Role of Same Host Agent

- Jenkins runs inside Docker
- Docker socket mounted:

`/var/run/docker.sock`

#### 24.9.1 Effect:

- Jenkins controls host Docker directly
  - Builds and pushes images without separate agent
- 

### 24.10 Observations

- Jenkins GUI simplifies CI/CD management
  - GitHub acts as source + pipeline definition
  - Docker ensures consistent builds
  - Webhook enables full automation
- 

### 24.11 Result

Successfully implemented a complete CI/CD pipeline where:

- Source code and pipeline are maintained in GitHub
  - Jenkins automatically detects changes
  - Docker image is built on host agent
  - Image is securely pushed to Docker Hub
- 

## 25 Experiment 9: Ansible Automation using Docker (SSH-based Multi-Server Setup)

### 25.1 Overview

This project demonstrates **configuration management and automation using Ansible** with multiple Docker containers acting as remote servers. It simulates a real-world infrastructure where Ansible manages multiple nodes via SSH.



---

## 25.2 Problem Statement

Manual server configuration leads to: - Configuration drift - Inconsistent environments - Time-consuming repetitive tasks

This experiment solves these issues using **Ansible automation**.

---

## 25.3 Technologies Used

- Ubuntu (WSL2)
  - Docker
  - Ansible
  - OpenSSH Server
  - SSH (password authentication)
  - Linux shell scripting
- 

## 25.4 Architecture

```
WSL (Control Node - Ansible)
|
| SSH (2201 - 2204)
↓
Docker Containers (Managed Nodes)
 server1
 server2
 server3
 server4
```

---

## 25.5 Setup Steps

### 25.5.1 1. Create Docker Image

```
docker build --no-cache -t ubuntu-server .
```

---

### 25.5.2 2. Run 4 Containers (Servers)

```
for i in {1..4}; do
 docker run -d --rm -p 220${i}:22 --name server${i} ubuntu-server
done
```

---

### 25.5.3 3. Verify Running Containers

```
docker ps
```

---

### 25.5.4 4. SSH Test

```
ssh root@localhost -p 2201
```

Password:

```
password
```

---

## 25.6 Inventory File (inventory.ini)

```
[servers]
server1 ansible_host=localhost ansible_port=2201
server2 ansible_host=localhost ansible_port=2202
server3 ansible_host=localhost ansible_port=2203
server4 ansible_host=localhost ansible_port=2204

[servers:vars]
ansible_user=root
ansible_password=password
ansible_connection=ssh
ansible_python_interpreter=/usr/bin/python3
```

---

## 25.7 Test Connectivity

### 25.7.1 Install dependency

```
sudo apt install sshpass -y
```

### 25.7.2 Run Ansible Ping

```
ansible all -i inventory.ini -m ping
```

---

## 25.8 Expected Output

```
server1 | SUCCESS => pong
server2 | SUCCESS => pong
```

```
server3 | SUCCESS => pong
server4 | SUCCESS => pong
```

---

## 25.9 Sample Playbook (update.yml)

```

- name: Configure servers
 hosts: servers
 become: yes

 tasks:
 - name: Update apt cache
 apt:
 update_cache: yes

 - name: Install packages
 apt:
 name:
 - vim
 - wget
 - htop
 state: present

 - name: Create test file
 copy:
 dest: /root/ansible_test.txt
 content: "Configured by Ansible on {{ inventory_hostname }}"
```

Run playbook:

```
ansible-playbook -i inventory.ini update.yml
```

---

## 25.10 Key Concepts Learned

- Ansible agentless architecture
  - SSH-based automation
  - Infrastructure as Code (IaC)
  - Docker-based virtualization
  - Inventory management
-

## 25.11 Common Issues & Fixes

### 25.11.1 1. SSH Permission Denied

- Ensure root password is set
- Enable PasswordAuthentication in sshd config

### 25.11.2 2. sshpass error

```
sudo apt install sshpass -y
```

### 25.11.3 3. Host key checking error

```
export ANSIBLE_HOST_KEY_CHECKING=False
```

---

## 25.12 Conclusion

This experiment successfully demonstrates how Ansible can automate configuration across multiple Docker-based servers using SSH, enabling scalable and consistent infrastructure management.

---